

Introduction to C

C Programming

- C: Uses a compiler to translate code into machine-native executable
- Execution is a two-step process:
 - Compile the program: `gcc my_program.c`
 - Run the executable: `./a.out`

Header Files in C

- Header files in C are used to declare the interfaces of functions, macros, and data types, allowing code to be shared and reused across multiple source files.
- Content:
 - Declares functions so they can be used across different .c files.
 - Defines macros that can be reused.
 - Declares structures, enums, and typedefs.
- Include
 - `#include <header.h>`: Used for system libraries.
 - `#include "header.h"`: Used for user-defined headers.

The printf() Function

- C uses `printf()` to display output, similar to Python's `print`
 - Format Strings: Use “%” followed by a character to format output (e.g., “%d” for integers, “%f” for floats)
 - Escape Characters: Use “\n” for newline, “\t” for tab, etc
 - Example:
 - `printf("# solns: %d\n", num_sol);`
 - Prints the text “# solns:”, the integer stored in `num_sol`, and a newline

Pointers

- C: Pointers store memory addresses, allowing direct access to variables and memory.
- Declaration: `int* p;` declares a pointer to an integer.
- Assignment: `p = &x;` stores the address of x in p.
- Dereferencing: `*p` accesses the value at the address stored in p
- Benefits: Enables dynamic memory allocation and efficient array handling.
- *Note: Be careful with memory management! Incorrect use of pointers can lead to memory leaks, segmentation faults, or other issues.*

Pointer Examples

```
#include <stdio.h>
int main()
{
    int x = 10;
    int *p = &x;
    // Outputs variable x
    printf("Value: %d\n", *p);
    // Outputs memory address of x
    printf("Address: %p\n", (void *)p);
    return 0;
}
```

```
y8441178@cycle2:~/678/lab0$ gcc test.c
y8441178@cycle2:~/678/lab0$ ./a.out
Value: 10
Address: 0x7ffd34d0a04c
```

Functions

- All executable code must be inside functions, unlike Python where top-level code can exist outside function.
- Functions cannot be nested. (e.g., you cannot declare a function inside another function).
- Main Function: The entry point of every C program is the main function, which typically returns int.
- Custom Functions: Define specific tasks within a program.
 - Function Types: Every function has a return type that must be specified (e.g., int, double, void).
 - Return Type: Specifies what type of value the function returns. Use void if the function doesn't return a value.

Function Calls

- Pass by Value: Function receives a copy of x and y.
 - Syntax: `result = add(x, y);`
 - Effect: Original values are unchanged.
- Pass by Reference (Pointers):
 - Syntax: `modify(&x);`
 - Effect: Function receives the address of x. Changes affect the original x.

Structs

- Structs allow programmers to define their own types and data structures

```
struct Example {  
    int x,y;  
};  
  
Struct Example exp = {10,10};
```

- Accessing Struct Members:
 - Dot (.) Operator: Used with struct variables.
 - `exp.x`
 - Arrow (->) Operator: Used with pointers to structs.
 - `exp->x`

Example

```
#include <stdio.h>

struct Point { //Defines a struct
    int x,y;
};

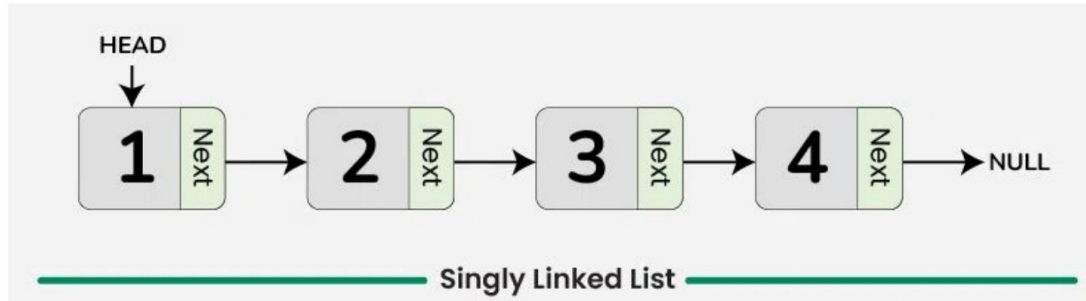
void movePoint(struct Point *p){ //Modify a struct using a pointer
    p -> x += 5;
    p -> y += 5;
}

int main(){
    struct Point pt = {10,10};
    movePoint(&pt);
    printf("Point: (%d, %d)\n", pt.x, pt.y); // Outputs (15, 15)
    return 0; }
```

Questions?

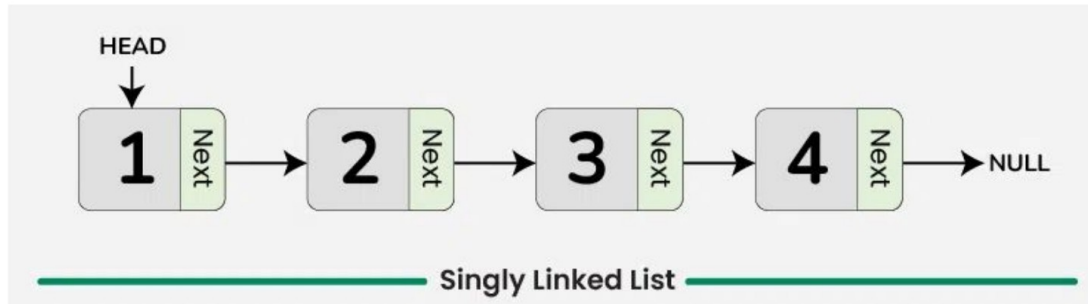
Linked List

- The C portion of this lab will be to implement a linked list data structure similar to python's list structure.
- Each node in the list will contain some data (an integer) and a pointer to the next element



How Linked Lists Work

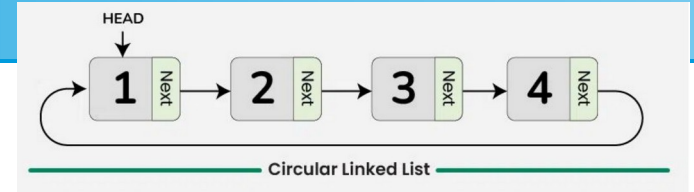
- **Head Node:** The starting point of the list
- **Traversal:** Move from one node to the next using the pointers until you reach a node where the next pointer is NULL (the end of the list)
- **Insertion/Deletion:** Nodes can be added or removed by changing the pointers



LinkedList.h

- Functions:
- **createNode**: Create a new node for the linked list with a given integer value.
- **insertAtEnd**: Add a new node with given data to the end of the linked list.
- **printList**: Traverse and print all the elements in the linked list.
- **deleteList**: Free all nodes in the linked list to prevent memory leaks.

ListQuestions.h



- **Task 1: Detecting Cycles in a Linked List**
 - Problem: If a node's next pointer eventually points back to a previous node, creating an infinite loop, a cycle exists.
 - Approach: Two-Pointer Technique (Fast&Slow): Use two pointers one moving faster than the other, If they meet a cycle is detected.
- **Task 2: Merging Two Sorted Linked Lists**
 - Problem: Merge two sorted linked lists into one sorted list without creating new nodes
 - Approach: Compare the nodes from both lists and link them in order. Continue merging by advancing pointers in the lists
 - This function should return the head of the newly merged list

Test Your Code

- We have provided a Makefile
- In the terminal
 - `make all` will compile all the code according to the rules in the makefile
 - `./runtests` runs the runTests.
 - If you've done everything correctly you should see 100/100
 - There is a tests directory that contains all 10 test cases.

Questions?

Debuggers

- Test cases are one way to see if your code works another way is to use a debugger
- Debuggers allow us to run code as disparate instructions rather than as an entire executable
- This works by creating additional symbol tables that enable tracking program behavior and relating it back to the source file

GNU Debugger (GDB)

- We will use GDB for this lab
- Capabilities
 - Execute a program
 - Stop the program using breakpoints
 - These can be conditional
 - Examine various types of information about the runtime environment
 - Value of any variable in the current scope
 - Condition of the runtime stack (call graph, stack frame, etc)
 - Condition of the hardware registers (stack pointer, frame pointer, program counter, etc)

GNU Debugger (GDB)

- Capabilities (cont)
 - Stop execution when a variable changes (watch points)
 - Call any function (that is in scope) at any point during execution
 - Examine the content of memory cells
 - Handle multiple source files

Compilation for Debugging

- Code must be compiled with the `-g` option
 - **Example:** `gcc -g -o main.c functions.c my_program.o`
- Debugging is always performed on the source files
- Optimizations must be disabled if the program is being compiled for debugging
 - Optimizations can remove variables, modify code organization and cause jumps in program execution while debugging

Questions?

Valgrind

- A heavyweight tool for dynamically catching hard-to-detect errors in programs
 - Only checks code that it runs
- Valgrind is a virtual machine and quite a bit slower than normally executing program in native environment
 - Translates binaries into an intermediate representation, performs some modifications and recompiles the transformations at runtime as needed
 - Allows it to instrument code with safe guards and profiling tools
- Valgrind's main tool is a memory checker that detects memory errors in programs
- Also has a few other useful tools such as cache and branch miss-prediction profilers, a heap profiler, a multi-thread data race detector, and more

Valgrind

- Memory checker (memcheck) helps find:
 - Memory leaks
 - Usage of uninitialized variables
 - Often the cause of non-deterministic behavior in single threaded software
- Bad frees of heap blocks
 - Double free
 - Freeing unallocated memory
- Accesses to unallocated memory
- Accesses to invalid stack and heap addresses
 - Freed heap addresses
 - Out of bounds heap addresses
 - Out of bounds stack addresses

valgrind_test.c

- This is a program that demonstrates all the ways valgrind can show how memory leaks occur
- This program exists for your benefit and we recommend trying it.

Questions?

Debugger Portion

- We have provided you with code that contains some bugs
- In this part of the lab you will use the debugger to find errors and correct the code
- We will walk you through the first two parts of the lab

A sample run

- From this we can see that mars somehow has bugs on it
- Also there is a segmentation fault which indicates improper memory management
- Lets throw Valgrind at it

```
j479r401@cycle4:~/temp/debuggers$ ./bugs
The current bug population of Earth is about: 14800000000000000000
The current bug population of Mars is about: 49152
The current bug population of Venus is about: 0
The total bug population of the solar system is: 14800000000000000000
The most useless bug is a butterfly Segmentation fault (core dumped)
```

Bug 1 (Valgrind)

- First we run
 - `valgrind ./bugs`
- The output mentions
 - uninitialised value(s)
 - `printf`
- This gives us a clue what is going on and Valgrind gives a recommended flag.

```
==2246023==
The current bug population of Earth is about: 1480000000000000000
==2246023== Use of uninitialised value of size 8
==2246023==    at 0x48E00BB: _itoa_word (_itoa.c:183)
==2246023==    by 0x48EAC9B: __printf_buffer (vfprintf-process-arg.c:155)
==2246023==    by 0x48EC73A: __vfprintf_internal (vfprintf-internal.c:1544)
==2246023==    by 0x48E11B2: printf (printf.c:33)
==2246023==    by 0x109418: main (bugs.c:108)
==2246023==
==2246023== Conditional jump or move depends on uninitialised value(s)
==2246023==    at 0x48E00CC: _itoa_word (_itoa.c:183)
==2246023==    by 0x48EAC9B: __printf_buffer (vfprintf-process-arg.c:155)
==2246023==    by 0x48EC73A: __vfprintf_internal (vfprintf-internal.c:1544)
==2246023==    by 0x48E11B2: printf (printf.c:33)
==2246023==    by 0x109418: main (bugs.c:108)
==2246023==
==2246023== Conditional jump or move depends on uninitialised value(s)
==2246023==    at 0x48EAD89: __printf_buffer (vfprintf-process-arg.c:186)
==2246023==    by 0x48EC73A: __vfprintf_internal (vfprintf-internal.c:1544)
==2246023==    by 0x48E11B2: printf (printf.c:33)
==2246023==    by 0x109418: main (bugs.c:108)
==2246023==
```

...

```
==2246023== Use --track-origins=yes to see where uninitialised values come from
```

Bug 1 (Valgrind)

- First we run
 - `valgrind --track-origins=yes ./bugs`
- This command informs us that the value was created on the stack rather than the heap.
- This is about all the lets look at the line numbers

```
The current bug population of Earth is about: 1480000000000000000
==2248164== Use of uninitialised value of size 8
==2248164==    at 0x48E00BB: _itoa_word (_itoa.c:183)
==2248164==    by 0x48EAC9B: __printf_buffer (vfprintf-process-arg.c:155)
==2248164==    by 0x48EC73A: __vfprintf_internal (vfprintf-internal.c:1544)
==2248164==    by 0x48E11B2: printf (printf.c:33)
==2248164==    by 0x109418: main (bugs.c:108)
==2248164== Uninitialised value was created by a stack allocation
==2248164==    at 0x109318: main (bugs.c:85)
```

Bug 1 (Valgrind)

- Line 85 is just the entry point to main
- printf.c is part of C if there is an error there then we have major problems with our C installation.
- Meaning this memory leak is almost certainly coming from line 108

```
The current bug population of Earth is about: 14800000000000000000
==2248164== Use of uninitialised value of size 8
==2248164==    at 0x48E00BB: _itoa_word (_itoa.c:183)
==2248164==    by 0x48EAC9B: __printf_buffer (vfprintf-process-arg.c:155)
==2248164==    by 0x48EC73A: __vfprintf_internal (vfprintf-internal.c:1544)
==2248164==    by 0x48E11B2: printf (printf.c:33)
==2248164==    by 0x109418: main (bugs.c:108)
==2248164== Uninitialised value was created by a stack allocation
==2248164==    at 0x109318: main (bugs.c:85)
```

Questions?

Bug 2 (GDB)

- To debug with GDB we run
 - `gdb ./bugs`
- Once in GDB
 - Type `r` (short for run) and hit enter
 - select `n` if asked for debuginfo
- This will run the executable and will pause it on segmentation faults

Bug 2 (GDB)

- We can now use the backtrace command to see a stack trace of the program
 - Type `bt` and hit enter
 - The output is a number of frames we can examine
- Note that once again the top most frames are part of C
- So we'll begin by looking at frame 4
 - Type `f 4` and hit enter

```
(gdb) f 4
#4  0x0000555555555243 in echo (strs=0x7fffffffe858) at bugs.c:53
53      printf("%s ", *strs);
```

Bug 2 (GDB)

- `printf()` expects a format string as it's first argument of type `char*` and we pass in `%s`.
- Because we've checked out this frame we can examine symbols in the frame using the `ptype` command
 - Type `ptype "%s"` and hit enter
 - This outputs `type = char [4]`
 - Which is how GDB represents an array of size 4 characters and that is what we want.
- Lets look at the second argument which should also be a `char*`
- If we run `ptype strs` we get `char **` and if we run `ptypes *strs` we get `char *`
- Once again what we were looking for

Bug 2 (GDB)

- The types are correct so lets now look at the values using the print command
 - Type `p strs` and hit enter
 - This returns a pointer
 - So we can dereference the pointer using `p *strs` and hit enter
 - This returns an error
- We can look around the error using pointer arithmetic
 - using `p *(strs-1)` returns “butterfly”
 - using `p *(strs+1)` returns 0x0, the null pointer

Bug 2 (GDB)

- We can use the list command to see the code that the frame is part of
 - Type `l` (lowercase L) and hit enter
 - You can use this command multiple times to show more lines of the code.
- The code keeps printing strings recursively until NULL is reached

```
(gdb) l
48      //
49      // An example of a NULL terminated array of strings is:
50      // char** strs = { "Hello", "World!", NULL };
51      void echo(char** strs) {
52          if (*strs != NULL) {
53              printf("%s ", *strs);
54              fflush(stdout);
55
56              echo(strs + 1); // Recurse on the next element in the string array
57          }
58      }
```

Bug 2 (GDB)

- Lets look at the final frame in the backtrace

```
(gdb) f 12
#12 0x000055555555475 in main (argc=1, argv=0x7fffffffe9a8) at bugs.c:123
123     echo bug_info.sentence);
```

- The next thing to check is what we passed into echo
 - `ptype bug_info` gives us the struct
 - `p bug_info` shows us the values
 - Note we can see “butterfly” but no NULL value

```
(gdb) ptype bug_info
type = struct Storage {
  intptr_t num_bugs_on_mars;
  const char *scary_bug;
  char *sentence[6];
  const char *colorful_bug;
  intptr_t num_bugs_on_earth;
  intptr_t num_bugs_on_venus;
  char *useless_bug;
}
```

```
(gdb) p bug_info
$6 = {num_bugs_on_mars = 0, scary_bug = 0x55555555601f "~~~~~ SPIDER!!! ~~~~~",
      sentence = {0x5555555592a0 "The", 0x5555555592c0 "most", 0x5555555592e0 "useless",
                  0x555555559300 "bug", 0x555555559320 "is", 0x555555559340 "a"},
      colorful_bug = 0x55555555600c "butterfly", num_bugs_on_earth = 1480000000000000000,
      num_bugs_on_venus = 0, useless_bug = 0x555555556016 "mosquito"}
```

Bug 2 (GDB)

- Since sentence doesn't have a NULL the echo function kept printing past "butterfly"
- Lets examine that first echo frame again
 - `p *strs` returns `0x148a04289b940000`
 - We can use typecasting and `p (intptr_t) *strs`
 - This returns `14800000000000000000` the number of bugs on earth
- So what we have is a classic buffer overflow error
- To fix this we can change the length of sentence to 7 and add `bug_info.sentence[6] = NULL;` when populating sentence

Questions?

Bug 3 (GDB)

- Lets recompile using `make` and run `./bugs` again

```
j479r401@cycle4:~/temp/debuggers$ ./bugs
The current bug population of Earth is about: 14800000000000000000
The current bug population of Mars is about: 0
The current bug population of Venus is about: 0
The total bug population of the solar system is: 14800000000000000000
The most useless bug is a mosquito
The most colorful bug is a butterfly
000/_] 000/_] p00/_] 00/_] @00/_] a is bug colorful most The ~~~~~ SPIDER!!! ~~~~~
The current bug adjective is: 0Jr0Z]
Segmentation fault (core dumped)
```

- We've uncovered more bugs so lets fix the gibberish after the word butterfly

Bug 3 (GDB)

- This bug occurs before the segmentation fault so we have to stop the code before the crash occurs
- We can pause the code's execution using breakpoints
 - Type `b main` and hit enter
- To see all breakpoints
 - Type `info breakpoints` and hit enter

```
(gdb) info breakpoints
Num    Type           Disp Enb Address                  What
1      breakpoint    keep y   0x00005555555532b in main at bugs.c:85
```

Bug 3 (GDB)

- However stopping at main isn't likely to be useful so lets also look at `bug_info.sentence` since it's being printed in the corrupted output
- To track when the program reads from `bug_info.sentence` we can use the `rwatch` watchpoint
 - Type `rwatch bug_info.sentence[0]` and hit enter
- There are other watchpoints such as `watch` which monitors when a value changes and `awatch` which monitors both read and writes
- Now lets continue running the code using the `continue` command
 - Type `c` and hit enter

Bug 3 (GDB)

- This will stop on echo which we know is working so lets skip ahead
 - Type `c 2` and hit enter
 - This will skip over our continue once and then stop on the second hit
 - We can continue using `c 2` until we reach the line containing gibberish

```
(gdb) c 2
Will ignore next crossing of breakpoint 2. Continuing.
0000000000000000 p00000000 00000000 @00000000
Hardware read watchpoint 2: bug_info.sentence[0]

Value = 0x5555555592a0 "The"
0x00000555555552f7 in echoohce (strs=0x7fffffff820) at bugs.c:78
78      for (; *iter != NULL; --iter)
```

- Here we can see the gibberish is getting printed in the echoohce function

Bug 3 (GDB)

- We can use backtrace (`bt`) to see what line `echoohce` is called on

```
(gdb) bt
#0  0x00005555555552f7 in echoohce (strs=0x7fffffffef820) at bugs.c:78
#1  0x00005555555554c8 in main (argc=1, argv=0x7fffffffef9a8) at bugs.c:135
```

- In this run it's on line 135 as indicated by the last frame line number
- So lets set a break point at line 135
 - Type `b bugs.c:135` and hit enter
- We can also remove our first two breakpoints using the delete `#` command
 - Type `d 1` and hit enter
 - Type `d 2` and hit enter
- Rerun the code using the `r` command

Bug 3 (GDB)

- The code will now stop on the echooce call

```
The most colorful bug is a butterfly
Breakpoint 3, main (argc=1, argv=0x7fffffffe9a8) at bugs.c:135
135     echoohce(bug_info.sentence);
```

- Lets use the next command to move onto the next instruction
 - Type `n` and hit enter

```
(gdb) n
00000000 00000000 p0000000 00000000 @00000000 a is bug colorful most The ~~~~~ SPIDER!!! ~~~~~
138     free(bug_info.sentence[0]);
```

- We've now completely jumped over echooce and are executing free()

Bug 3 (GDB)

- Rerun your code again using `r`
- This time we'll go inside the function using the step command
 - Type `s` and hit enter

```
Breakpoint 3, main (argc=1, argv=0x7fffffff9a8) at bugs.c:135
135     echohce(bug_info.sentence);
(gdb) s
echohce (strs=0x7fffffff820) at bugs.c:64
64     char** stop_beginning = strs - 1; // HINT: What could this possibly be used for?
```

Bug 3 (GDB)

- Once inside we can use the next command (`n`) to execute the next instruction
- Then hit enter twice more
- You'll notice even though we went over a `printf` nothing was written to output.
- `Printf` is buffered so we need to flush the buffer before we can see it
 - Type `call fflush(stdout)` and hit enter

```
(gdb) n
67     for (iter = strs; *iter != NULL; ++iter)
(gdb)
68         printf("%s ", iter);
(gdb)
67     for (iter = strs; *iter != NULL; ++iter)
(gdb) call fflush(stdout)
UUUUUU Couldn't write extended state status: Bad address.
```


Bug 3 (GDB)

- Now that we're in the right location we can begin by making sure our types are correct again

```
(gdb) ptype iter
type = char **
(gdb) p iter
$1 = (char **) 0x7fffffffef820
(gdb) p *iter
$2 = 0x5555555592a0 "The"
```

- The problem is that iter is not dereferenced
- Lets fix that and rerun the code